

Deep Research: Avatrada 57 Topic 049

Engineering Report: PostgreSQL 15 Asynchronous Streaming Replication

TO: Engineering Team **FROM:** Autonomous Technical Researcher **DATE:** October 26, 2023 **SUBJECT:** Deep-Dive on PostgreSQL 15 Asynchronous Replication for Cross-Region Disaster Recovery

Executive Summary

This report provides a detailed technical analysis of PostgreSQL 15's asynchronous streaming replication, as specified for creating a real-time replica of the Ledger database in a failover region (`us-east1`). The primary objective is to ensure business continuity by maintaining a hot standby that can be promoted to a primary server in the event of a regional outage.

We will deconstruct the architecture of streaming replication, provide a step-by-step implementation guide, detail critical monitoring and failover procedures, and analyze potential failure modes and optimizations. The analysis confirms that asynchronous replication is the appropriate choice for cross-region disaster recovery due to its low-performance impact on the primary, but it necessitates rigorous monitoring of replication lag to manage the Recovery Point Objective (RPO).

1. Technical Deconstruction

PostgreSQL's streaming replication is a mechanism for copying database changes from a primary server to one or more standby servers in near real-time. It is built upon the Write-Ahead Logging (WAL) protocol, which is PostgreSQL's standard mechanism for ensuring data durability.

Core Architecture & Data Flow

1. **Primary Server:** The main database server that accepts both read and write connections. All data modifications are first written to a WAL file before being applied to the database's data files (heap).
2. **WAL (Write-Ahead Log):** An ordered sequence of records describing every change made to the database. This log is the source of truth for replication.
3. **WAL Sender Process:** A dedicated process on the primary server that reads WAL records from disk and "streams" them over the network to a connected standby server.
4. **Standby Server (Replica):** A server that connects to the primary and receives the WAL stream. It runs in a continuous "recovery mode."
5. **WAL Receiver Process:** A process on the standby server that receives the WAL stream from the primary's WAL Sender and writes it to the standby's own WAL files.
6. **Startup/Replay Process:** A process on the standby that reads the WAL records written by the WAL Receiver and applies the described changes to its own data files, thereby keeping it in sync with the primary.

Asynchronous vs. Synchronous Replication

The key difference lies in when the primary server considers a transaction committed.

- **Asynchronous (Specified):** The primary server commits a transaction as soon as the WAL record is written to its *local* disk. It does not wait for confirmation from the standby.
 - **Pros:** Minimal to no performance impact on the primary's write throughput, tolerant of high network latency (ideal for cross-region).
 - **Cons:** Potential for data loss. If the primary fails before a committed transaction's WAL record has been sent and applied to the standby, that transaction is lost upon failover. The amount of potential data loss is equal to the replication lag.

- **Synchronous:** The primary waits for confirmation from at least one standby that the WAL record has been received and written to disk before confirming the commit to the client.
 - **Pros:** Guarantees zero data loss (RPO=0) if the standby is available.
 - **Cons:** Write performance is limited by the round-trip network latency to the standby. This is prohibitive for cross-region setups.

For the specified use case (failover to `us-east1`), asynchronous replication is the correct architectural choice.

2. Implementation Strategy

This section provides a concrete guide for setting up a PostgreSQL 15 primary and a hot standby in a different region.

Assumptions: * Primary Server IP: `10.10.0.10` (e.g., us-west-2) * Standby Server IP: `10.20.0.20` (e.g., us-east-1) * PostgreSQL 15 is installed on both servers. * Network connectivity and firewall rules (port `5432`) are established between the servers.

Step 1: Configure the Primary Server

1. **Create a Replication User:** This dedicated user will be used by the standby to connect. `sql -- Connect to the primary via psql CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'a_very_secure_password';`
2. **Configure `pg_hba.conf`:** Allow the replication user to connect from the standby's IP address. Add this line to the *top* of the file. `ini # /var/lib/postgresql/15/data/pg_hba.conf # TYPE DATABASE USER ADDRESS METHOD host replication replicator 10.20.0.20/32 scram-sha-256`
3. **Configure `postgresql.conf`:** Enable replication settings. ```ini # /var/lib/postgresql/15/data/postgresql.conf`

`listen_addresses = '*' # Listen on all network interfaces wal_level = replica`
`# Minimum level for replication max_wal_senders = 5 # Number of`

concurrent replication connections wal_keep_size = 1024 # In MB. Min size of WAL files to keep for standbys. # Consider using replication slots for better reliability. ``

4. **Restart the Primary Server** to apply the changes. `bash sudo systemctl restart postgresql-15`

Step 2: Clone the Primary to the Standby

The most reliable way to initialize a standby is using `pg_basebackup`. This tool copies the primary's entire data directory while ensuring it's in a consistent state.

1. **Stop PostgreSQL on the Standby** if it's running.
2. **Clear the Standby's Data Directory.** `bash sudo rm -rf /var/lib/pgsql/15/data/*`
3. **Run `pg_basebackup` from the Standby Server:** ```bash # Run as the 'postgres' user pg_basebackup -h 10.10.0.10 -p 5432 -U replicator -D /var/lib/pgsql/15/data/ \ -Fp -Xs -R -P`

Flag breakdown:

-h: Primary's host

-p: Primary's port

-U: Replication username

-D: Standby's data directory

-Fp: Format plain (not tar)

-Xs: Stream WAL files while the backup is running

-R: Create standby configuration files (standby.signal and postgresql.auto.conf)

-P: Show progress

...

Step 3: Verify Standby Configuration

The `-R` flag in `pg_basebackup` automatically handles the standby's configuration.

1. **`standby.signal` file:** A zero-byte file named `standby.signal` will be created in the data directory. Its presence tells PostgreSQL to start in recovery/standby mode.
2. **`postgresql.auto.conf`:** This file will be populated with the connection info for the primary. This directly answers the prompt's question.

`primary_conninfo` Configuration: The content of `/var/lib/pgsql/15/data/postgresql.auto.conf` will be: ```ini

Do not edit this file manually!

It will be overwritten by the ALTER SYSTEM command.

```
primary_conninfo = 'user=replicator password=a_very_secure_password
host=10.10.0.10 port=5432 sslmode=prefer sslcompression=0
gssencmode=prefer krbsrvname=postgres target_session_attrs=any' ```
```

This line instructs the standby on how to connect to its primary.

Step 4: Start and Verify the Standby

1. **Start the PostgreSQL service on the standby.** `bash sudo systemctl start postgresql-15`
2. **Check Logs on the Standby:**
`bash sudo journalctl -u postgresql-15 -f` # Look for messages like: #
LOG: started streaming WAL from primary at ... # LOG: consistent
recovery state reached at ... # LOG: database system is ready to accept
read-only connections

3. **Check Replication Status on the Primary:** `sql -- Connect to the primary via psql SELECT client_addr, state, sync_state, pg_wal_lsn_diff(sent_lsn, write_lsn) as write_lag_bytes, pg_wal_lsn_diff(sent_lsn, flush_lsn) as flush_lag_bytes, pg_wal_lsn_diff(sent_lsn, replay_lsn) as replay_lag_bytes FROM pg_stat_replication;` This query should show one row for the connected standby (10.20.0.20), with `state = streaming` and `sync_state = async` .

3. Monitoring and Failover Procedures

How to Monitor 'Replication Lag'

Replication lag is the measure of how far behind the standby is from the primary. It can be measured in bytes or in time.

Method 1: Byte-based Lag (Most Accurate)

Query `pg_stat_replication` on the **primary** server. This shows the difference in Log Sequence Numbers (LSN), which is a pointer to a location in the WAL.

```
-- Run on PRIMARY
SELECT
    client_addr,
    application_name,
    state,
    pg_wal_lsn_diff(pg_current_wal_lsn(), sent_lsn) AS sent_lag_bytes,
    pg_wal_lsn_diff(sent_lsn, replay_lsn) AS replay_lag_bytes
FROM pg_stat_replication;
```

- `sent_lag_bytes` : How many bytes the primary has written that have not yet been sent to the standby. This is typically very low unless the network is saturated.
- `replay_lag_bytes` : How many bytes the standby has received but has not yet applied. This can grow if the standby is under heavy I/O load. The total lag is the sum of these two.

Method 2: Time-based Lag (Easier to Interpret)

Query the standby server to find the timestamp of the last transaction it replayed.

```
-- Run on STANDBY
SELECT
  CASE
    WHEN pg_last_wal_receive_lsn() = pg_last_wal_replay_lsn() THEN '0
seconds'
    ELSE (now() - pg_last_xact_replay_timestamp())::text
  END AS replication_lag;
```

Caveat: This timestamp only updates when a transaction with a timestamp is replayed. On an idle system, this value can grow misleadingly. It is most useful on a system with constant write activity.

How to Promote the Standby to Primary

In a failover scenario where the primary is confirmed to be down and unrecoverable, the standby must be promoted.

Promotion Command: The standard tool for this is `pg_ctl`.

1. **SSH into the Standby Server.**
2. **Execute the promote command** as the `postgres` user.

```
```bash
```

## Ensure you know the path to your data directory

---

```
pg_ctl -D /var/lib/pgsql/15/data/ promote ```
```

**What Happens During Promotion:** 1. The `pg_ctl promote` command creates a `promote.signal` file in the data directory. 2. The standby's recovery process sees



this file, finishes replaying any outstanding WAL in its possession, and then exits recovery mode. 3. The `standby.signal` file is removed. 4. The server timeline is incremented, and it begins accepting read-write connections, effectively becoming the new primary.

**Post-Promotion Actions:** 1. **Update Application DNS/Connection Strings:**

All applications must be redirected to the new primary's IP address. 2. **Fence the Old Primary:** The old primary server must be prevented from coming back online and accepting writes, which would cause a "split-brain" scenario. This can be done via network ACLs, shutting down the VM (STONITH - Shoot The Other Node In The Head), or other infrastructure-level controls. 3. **Rebuild the Old Primary:** Once the old primary server is accessible, it must be re-synced as a new standby to the *new* primary, typically using `pg_rewind`.

---

## 4. Critical Analysis

### Potential Failure Modes

1. **Network Partition:** If the network link between regions fails, the standby will stop receiving WAL updates. The replication lag will grow indefinitely. The primary will continue to operate normally, but the RPO (Recovery Point Objective) will degrade with every new transaction.
  - **Mitigation:** Robust network monitoring and alerting on replication lag exceeding a defined threshold (e.g., > 5 minutes).
2. **Split-Brain:** This is the most dangerous scenario. It occurs if the old primary comes back online after a failover and applications reconnect to it. You now have two independent, writeable masters, leading to inconsistent and divergent data.
  - **Mitigation:** Strict operational procedures for failover. Implement fencing (STONITH) to ensure the old primary is powered off before the standby is promoted. Use a load balancer or connection pooler that can be quickly reconfigured to point to the new primary.

3. **Standby Performance Bottleneck:** If the standby server has insufficient I/O, CPU, or memory resources, it may not be able to replay WAL records as fast as they are received. This causes the `replay_lag` to grow, increasing RPO even with a healthy network.

- **Mitigation:** Provision the standby with hardware equivalent to the primary. Monitor replay lag specifically.

4. **WAL Buildup on Primary:** If the standby is offline for an extended period, the primary will retain WAL files for it. If using `wal_keep_size`, this can fill up the primary's disk, potentially causing a database outage.

- **Mitigation:** Use **Replication Slots**.

## Edge Cases & Optimizations

1. **Replication Slots:** A replication slot is a feature that guarantees the primary will not remove WAL segments until they have been consumed by the standby.

- **Advantage:** Far more reliable than `wal_keep_size`. It prevents WAL buildup on the primary from causing an outage *if the standby is online*. However, if the standby is permanently gone, the slot will cause WAL to accumulate indefinitely until it is manually dropped.
- **Implementation:** Create a slot on the primary ( `SELECT pg_create_physical_replication_slot('standby_east1');` ) and configure `primary_slot_name = 'standby_east1'` on the standby.

2. **Failover Automation:** Manual promotion with `pg_ctl` is error-prone under pressure.

- **Optimization:** Employ a dedicated cluster management tool like **Patroni** or **repmgr**. These tools automate the failover process, including leader election, standby promotion, and fencing, significantly reducing the risk of human error and minimizing downtime (RTO - Recovery Time Objective).

3. **Read-Only Queries on Standby:** A hot standby ( `hot_standby = on` in `postgresql.conf` , which is the default) can be used for read-only queries. This can offload reporting and analytical workloads from the primary.

- **Caveat:** Long-running queries on the standby can conflict with WAL replay, further increasing replication lag. Configure `max_standby_streaming_delay` to manage this trade-off.